

AI Meeting Assistant

Layihə bələdçisi və müəllim təqdimatı planı — Month-4 Capstone

Samir Nəbiyev · Canlı sistem: <https://api.visualkey.az/ui/index.html> · Kod: github.com/nebiyevsamir002-star/meeting-assistant

1. Layihə bir baxışda

Onlayn iclaslarda qeyd götürmə ehtiyacını aradan qaldıran AI köməkçi. İstifadəçi söhbətə fokuslanır — sistem qalan hər şeyi edir. Layihə DigitalOcean serverində istehsal rejimində işləyir (HTTPS, real Claude + ElevenLabs + OpenAI açarları ilə).

Üç mərhələ:

- 1. İclasdan əvvəl** — sənədlər (vəb, PDF, OCR) n8n workflow-u ilə yığılır, parçalanır, embedding-lənir və Qdrant bilik bazasına yüklənir. Google Calendar-dan yaxın iclaslar APScheduler ilə avtomatik hazırlanır.
- 2. İclas zamanı** — Mac-dəki agent sistem səsini (BlackHole) 20 saniyəlik parçalarla ElevenLabs Scribe STT-yə göndərir. Claude sualları aşkarlayır, ReAct agenti bilik bazasında axtarış aparıb 3 tonda cavab variantı təklif edir. Canlı feed UI-da axır.
- 3. İclasdan sonra** — LangGraph pipeline-ı transkript, strukturlaşdırılmış xülasə və action item-lər hazırlayır, epizodik yaddaşa (Qdrant + mem0) yazır və hesabatı n8n vasitəsilə email + Telegram-a çatdırır.

Texnologiyalar: Python 3.11 · FastAPI · SQLite · APScheduler · Claude API · OpenAI · Instructor · LangChain (ReAct, memory) · LangGraph · Qdrant · mem0 · ElevenLabs · n8n · htmlcsstoimage · Docker · nginx · Certbot · GitHub Actions CI/CD · DigitalOcean

2. Fayl bələdçisi — hər fayl nə üçündür

Kök qovluq — sənədlər və konfigurasiya

Fayl	Nə üçündür
README.md	Giriş sənədi: layihənin nə etdiyi, sürətli başlanğıc, arxitektura sxemi, qovluq strukturu.
PLAN.md	Hər fazanın Month-4 kurs mövzusunə xəritəsi + yaddaş arxitekturası cədvəli. Müəllim üçün əsas sənəd.
SETUP.md	Bütün API açarlarının/hesabların haradan alınacağı, addım-addım (Anthropic, ElevenLabs, Google OAuth, HCTI, SMTP, Telegram, OCR.space, BlackHole).
DEPLOY.md	DigitalOcean deployment runbook-u: droplet seçimi, Termius, Docker, HTTPS, n8n, CI/CD.
Makefile	Qısa komandalar: make install / run / test / smoke / compose-up.
requirements.txt	Python asılılıqları (FastAPI, LangChain, LangGraph, Qdrant, mem0, Instructor və s.).
requirements-dev.txt	Test asılılığı (pytest).
.env.example	Bütün mühit dəyişənlərinin şablonu — sirlərsiz. Hər provayderin mock/real keçidi burada.
.gitignore / .dockerignore	Git-ə və Docker image-inə düşməməli fayllar (.env, data/, .venv).
Dockerfile	İnkişaf üçün Docker image (Python 3.11-slim).
docker-compose.yml	Lokal stack: app + Qdrant + n8n bir komanda ilə (docker compose up).

app/ — əsas tətbiq paketi

Fayl	Nə üçündür
main.py	FastAPI giriş nöqtəsi: lifespan (DB init, APScheduler start), router-lər, UI mount.

Fayl	Nə üçündür
config.py	pydantic-settings — bütün konfigurasiya .env-dən tipli oxunur; sirlər kodda yoxdur.
prompts.py	Bütün LLM prompt şablonları bir yerdə (sual aşkarlama, xülasə, action, entity).
runtime.py	Provayder singleton-ları (lru_cache factory-lər) — bütün tətbiq eyni instansı paylaşır.

app/api/ — REST endpointlər

Fayl	Nə üçündür
__init__.py	Bütün router-lərin birləşdirilməsi.
deps.py	Autentifikasiya asılılığı: JWT Bearer + daxili servislər üçün X-API-Key.
health.py	/health — status + hansı provayderlərin aktiv olduğu.
auth.py	JWT token verilməsi + Google OAuth 2.0 axını (login/callback endpointləri).
ingest.py	Bilik bazası: n8n-dən sənəd qəbulu, PDF/mətn yükləmə, semantik axtarış.
meetings.py	İclas CRUD, mətn/audio seqmentlər, canlı feed (polling), bitirmə + hesabat.
memory_api.py	Keçmiş iclaslarda axtarış (epizodik Qdrant + mem0).
calendar_api.py	Yaxın təqvim hadisələri + planlayıcının əl ilə işə salınması.

app/providers/ — dəyişdirilə bilən provayderlər (hamısının mock-u var)

Fayl	Nə üçündür
llm/base.py	LLM interfeysi: complete() + generate_structured() — bütün provayderlər üçün ortaq.
llm/mock.py	Açırsız işləyən mock: ReAct formatını və structured cavabları qaydalarla imitasiya edir.
llm/claude.py	Anthropic Claude: chat + Instructor/tool-use ilə structured output.
llm/openai_llm.py	OpenAI — Claude-un tam əvəzləyicisi (LLM_PROVIDER=openai ilə keçid).
stt/...	Səs→mətn: elevenlabs.py (Scribe REST), whisper.py (OpenAI), mock.py (ssenarili transkript).
tts.py	Mətn→səs (istəyə bağlı): ElevenLabs / OpenAI / mock.
embeddings.py	Mətn→vektor: OpenAI (dimensions=256) / deterministik hash mock.
vector.py	Vektor bazası abstraksiyası: Qdrant (real) + in-memory (mock), auto-fallback.
calendar/google.py	Google Calendar OAuth 2.0 REST inteqrasiyası (token saxlama + refresh).
card.py	Xülasə kartı: htmlcsstoimage API / lokal HTML mock.
delivery.py	Hesabat çatdırılması: n8n webhook / lokal outbox mock.

app/agent/ — LangChain ReAct agenti

Fayl	Nə üçündür
llm_adapter.py	Bizim provayder abstraksiyasını LangChain-ə 'tanıdan' körpü sinfi.
tools.py	StructuredTool-lar (Pydantic args_schema): search_context, get_recent_transcript, detect_question.
react_agent.py	ReAct AgentExecutor qurulması: Thought→Action→Observation dövrü, max_iterations, intermediate steps.

app/memory/ — dörd yaddaş qatı

Fayl	Nə üçündür
short_term.py	LangChain Window (son K seqment) + Summary (artan xülasə) yaddaşı.

Fayl	Nə üçündür
entity.py	Varlıq yaddaşı: şəxslər/layihələr/tarixlər structured output ilə yığılır.
long_term.py	Uzunmüddətli yaddaş: mem0 (real, OpenAI+Qdrant) / SQLite mock.
episodic.py	Epizodik yaddaş: hər iclas Qdrant-da vektorlaşdırılmış epizod kimi.

app/ingestion, services, models, storage, static

Fayl	Nə üçündür
ingestion/chunker.py	Mətni cümlə sərhədli, üst-üstə düşən parçalara bölür.
ingestion/service.py	Sənəd → parçalar → embedding → Qdrant; semantik axtarış funksiyası.
services/meeting_service.py	CANLI PIPELINE (sistemin ürəyi): seqment → sual aşkarlama → ReAct agent → cavab variantları → feed.
services/post_meeting.py	İclas-sonrası LangGraph StateGraph + MemorySaver: transkript→xülasə→action→yaddaş→kart→çatdırılma.
services/scheduler.py	APScheduler: təqvim dövrü yoxlayıb yaxın iclasları hazırlayır.
models/core.py	Domen Pydantic modelləri: Meeting, TranscriptSegment, FeedEvent, MeetingReport...
models/structured.py	LLM structured output sxemləri: DetectedQuestions, AnswerOptions, ActionItems...
storage/db.py + repo.py	SQLite sxemi və təmiz SQL repository funksiyaları (ORM-siz, oxunaqlı).
static/index.html	4 addımlı istifadəçi interfeysi: Yarat→Başlat→Canlı→Hesabat + sənəd yükləmə + login.

capture/ — Mac tərəfi (səs tutma)

Fayl	Nə üçündür
agent.py	AVTOMATİK rejim: LaunchAgent daemonu — serveri izləyir, iclas canlı olan kimi BlackHole-dan yazır, bitəndə dayanır.
install_agent.sh / uninstall_agent.sh	Agentin bir-dəfəlik quraşdırılması (config + LaunchAgent + mikrofon icazəsi) və silinməsi.
capture_blackhole.py	Əl rejimi: BlackHole-dan 20s parçalarla yazıb API-yə göndərir.
send_file.py	Fayl rejimi: hazır WAV faylını parçalayıb göndərir — BlackHole-suz test.
requirements.txt	Yalnız capture üçün: sounddevice, numpy.

n8n/ — vizual avtomatlaşdırma workflow-ları

Fayl	Nə üçündür
scrape_ingest_workflow.json	Scraping pipeline: Webhook → mənbə növünə görə ayırma (vəb HTML-extract / PDF extract / OCR.space) → təmizləmə → X-API-Key ilə /api/ingest-ə POST.
delivery_workflow.json	Çatdırılma: Webhook → mesaj formatı (Code node) → paralel Email (SMTP) + Telegram; bir kanal xəta versə digəri davam edir (onError=continue).

deploy/ + .github/ — istehsal infrastrukturu

Fayl	Nə üçündür
deploy/Dockerfile.prod	Multi-stage istehsal image-i, qeyri-root istifadəçi (təhlükəsizlik).
deploy/docker-compose.prod.yml	İstehsal stack-i: app + Qdrant + n8n + nginx + certbot.
deploy/nginx/meeting-assistant.conf	Reverse proxy: api.visualkey.az → app, n8n.visualkey.az → n8n, HTTP→HTTPS.
deploy/activate_https.sh	Bir komanda ilə HTTPS: DNS yoxlaması → Let's Encrypt sertifikatı → prod stack.
.github/workflows/ci.yml	CI/CD: hər push-da pytest → kod serverə (scp) → Docker rebuild. Secrets ilə parollu SSH.

tests/ + scripts/ — keyfiyyət təminatı

Fayl	Nə üçündür
tests/conftest.py	Test mühiti: mock provayderlər, müvəqqəti SQLite, TestClient.
tests/test_meeting_flow.py	Uçdan-uca test: ingest→ıclas→sual→cavab→hesabat→yaddaş (ən vacib test).
tests/test_providers.py	Mock LLM/STT/embedding davranış testləri (ReAct axını daxil).
tests/test_auth.py / test_health.py / test_chunker.py	JWT, health endpointi və parçalama testləri. Cəmi 15 test.
scripts/smoke_test.py	Canlı smoke test: real HTTP ilə bütün mərhələləri yoxlayır (lokal və ya prod serverə qarşı).

3. Təqdimat planı — 25-30 dəqiqəlik ssenari

Qızıl qayda: əvvəlcə NƏTİCƏNİ göstər (canlı demo), sonra NECƏSİNİ izah et (kod), sonunda NƏYİ ÖYRƏNDİYİNİ bağla (kurs mövzuları + testlər). Aşağıdakı ardıcılıq bu prinsipə qurulub.

1. Giriş — problem və ideya · 2 dəq · [README.md](#)

«İclasda həm dinləyib, həm qeyd aparmaq mümkün deyil. Mən elə sistem qurdum ki, insan söhbətə fokuslanır, AI isə qeydləri aparır, sualları tutur və cavab təklif edir.» README-dəki arxitektura sxemini göstərin.

2. CANLI DEMO — ən güclü hissə · 5 dəq · <https://api.visualkey.az/ui/index.html>

Telefonda/brauzerdə: iclas yaradın → sənəd yükləyin (bilik bazası) → Başlat → Meet-də danışın (Mac agentı avtomatik yazır) və ya «Nümunə cümlə göndər» → sualın aşkarlanması və 3 tonlu cavab variantlarını göstərin → Bitir → hesabın ekranda, EMAILDƏ və TELEGRAMDA gəlməsini göstərin. Bu 5 dəqiqə qiymətin yarısıdır.

3. Kurs mövzuları xəritəsi · 2 dəq · [PLAN.md](#)

Cədvəli açıb deyin: «Month-4-ün bütün mövzuları bu layihədə real istifadə olunub» — hər sətiri barmaqla göstərin.

4. Konfigurasiya və provayder abstraksiyası · 4 dəq · [config.py](#) → [providers/llm/](#)

«Bir env dəyişəni ilə Claude↔OpenAI keçir, açar yoxdursa mock işləyir» — base.py interfeysi, mock.py fəlsəfəsi, claude.py-da Instructor/structured output. Bu, layihənin ən güclü mühəndislik qərarıdır.

5. ReAct agent və alətlər · 4 dəq · [agent/tools.py](#) → [react_agent.py](#)

StructuredTool + Pydantic args_schema göstərin; react_agent.py-da Thought→Action→Observation formatını izah edin. UI-dakı «agent addımları» ilə birləşdirin: «demoda gördüyünüz cavab bu dövrün nəticəsi idi».

6. Canlı pipeline və yaddaş qatları · 4 dəq · [services/meeting_service.py](#) → [memory/](#)

process_text_segment() axını yuxarıdan aşağı oxuyun. Sonra 4 yaddaş qatını sadalayın: Window+Summary (LangChain), Entity, mem0 (uzunmüddətli), Qdrant epizodik.

7. İclas-sonrası LangGraph · 2 dəq · [services/post_meeting.py](#)

StateGraph qovşaqlarını göstərin: assemble→summarize→actions→memorize→card→deliver; MemorySaver checkpoint-i qeyd edin.

8. n8n workflow-ları · 3 dəq · <https://n8n.visualkey.az> (kanvas)

İki workflow-u KANVASDA açın: scraping (web/PDF/OCR budaqları) və delivery (Email+Telegram). «FastAPI iclas bitəndə webhook-a POST edir, qalanını n8n edir» — vizual olduğu üçün çox effektivdir.

9. Mac agentı (avtomatlaşdırma) · 2 dəq · [capture/agent.py](#)

«Heç kim iclas başlayanda terminal açmaq istəmir — ona görə LaunchAgent daemonu yazdım: serveri izləyir, iclas canlı olan kimi özü yazmağa başlayır.» Logu göstərin: tail -f ~/Library/Logs/meeting-assistant-capture.log

10. İstehsal infrastrukturu · 3 dəq · [deploy/](#) + [GitHub Actions](#)

docker-compose.prod.yml (nginx+certbot), sonra GitHub-da Actions tabını açın: «hər push avtomatik test olunur və serverə çıxır». Yaşıl run-ları göstərin. HTTPS sertifikatının avtomatik yeniləndiyini deyin.

11. YEKUN — keyfiyyət sübutu · 2 dəq · [tests/](#) + [scripts/smoke_test.py](#)

Terminalda make test (15 test) və istəsəniz canlı smoke testi işlədin. Son cümlə: «Sistem sıfır açarla da işləyir (mock), açarlar yalnız onu gücləndirir — bu, test edilə bilən arxitekturanın nəticəsidir.»

Xülasə: haradan başla, harada bitir

İlk açılan: README.md (problem) → dərhal CANLI DEMO. **Kod turunun nüvəsi:** config.py → providers → agent → meeting_service.py → post_meeting.py. **Vizual fasilə:** n8n kanvasları. **Son açılan:** tests/ + smoke_test — «işlədiyini sübut edirəm» notu ilə bitirin. PLAN.md-ni çap edib müəllimə VERİN — qiymətləndirmə xəritəsi hazır olur.

4. Demo öncəsi hazırlıq (təqdimatdan 30 dəq əvvəl)

- [+] `curl https://api.visualkey.az/health` — «ok» və `llm=claude, stt=elevenlabs` görünməlidir.
- [+] UI-a daxil olun (samir / şifrəniz) — köhnə test iclaslarının arasında itməmək üçün mövzu adlarını tanıyın.
- [+] `n8n`-də hər iki workflow-un Active olduğunu yoxlayın (yaşıl açar).
- [+] Telegram botunuzun söhbəti açıq olsun — hesabat gələndə dərhal göstərə bilərsiniz.
- [+] Mac agenti işləyir? → `launchctl list | grep meeting-assistant` (PID görünməlidir).
- [+] Səs çıxışını Multi-Output Device-a keçirin (həm eşidirsiniz, həm BlackHole yazır).
- [+] Bilik bazasına 1-2 real sənəd yükləyin (məs. layihə brifi) — demo sualına əsaslı cavab çıxsın.
- [+] Ehtiyat plan: internet zəif olsa, lokalda «make run» + mock rejimi eyni axını göstərir.

5. Müəllimin verə biləcəyi suallar — hazır cavablar

S: Niyə hər provayderin mock versiyası var?

Üç səbəb: (1) açarsız inkişaf və testlər — `pytest` heç bir xarici xidmətə toxunmur; (2) API xərcinə nəzarət; (3) abstraksiyanın düzgünlüyünü sübut edir — mock ilə işləyən kod real provayderlə də işləyir.

S: Claude embedding vermir — semantik axtarışı necə qurdun?

Embedding üçün OpenAI `text-embedding-3-small` istifadə edirəm (`dimensions=256` parametri ilə), LLM üçün Claude. Provayderlər ayrı abstraksiyalardadır, ona görə qarışdırmaq mümkündür.

S: ReAct nədir və niyə sadə prompt yox?

ReAct = Reasoning + Acting: model düşünür (Thought), alət çağırır (Action), nəticəni görür (Observation) və dövrü təkrarlayır. Sadə promptdan fərqi: cavab bilik bazasındakı FAKTLARA əsaslanır, uydurma azalır. `intermediate_steps`-i UI-da göstərirəm — şəffaflıq.

S: Yaddaş qatlarının fərqi nədir?

Window — son K seqment (ani kontekst); Summary — bütün iclasın artan xülasəsi; Entity — kim/nə xatırlandı; Epizodik (Qdrant) — «keçən iclasda nə danışdıq» vektor axtarışı; `mem0` — iclaslar arası faktlar (LLM özü fakt çıxarır).

S: LangGraph-ı nə üçün istifadə etdin?

İclas-sonrası 6 addımlı pipeline StateGraph kimi qurulub; MemorySaver checkpointer hər addımın vəziyyətini saxlayır — addım xəta versə haradan davam edəcəyi bellidir.

S: Təhlükəsizlik nə səviyyədədir?

JWT autentifikasiya (UI), X-API-Key (daxili `n8n`→API), HTTPS (Let's Encrypt, avto-yenilənən), nginx reverse proxy, ufw yalnız 22/80/443, fail2ban brute-force qorunması, qeyri-root Docker konteyner, sirlər yalnız `.env`-də.

S: 2GB serverə bu qədər servis necə sığır?

Swap faylı + yüngül seçimlər: SQLite (ayrıca DB serveri yox), Qdrant kiçik kolleksiyalarla, nginx alpine. Pik istifadədə 4GB plana keçmək bir droplet resize-dır.

S: Real vaxtda gecikmə nə qədərdir?

Audio 20 saniyəlik parçalarla gedir; STT + sual aşkarlama + agent ~3-6 saniyə əlavə edir. Yeni sual səsəndikdən ~25 saniyə sonra cavab variantları ekrandadır.

S: Bu layihənin aylıq xərci nə qədərdir?

Server \$12/ay + domen ~\$10/il + API-lər istifadəyə görə (1 saatlıq iclas ~\$0.5-1). Tələbə üçün tam əlçatandır.

S: Nəyi fərqli edərdin / gələcək planlar?

WebSocket ilə polling-siz canlı feed; speaker diarization (kim danışır); iclas video platformalarına birbaşa bot kimi qoşulma; çoxistifadəçili hesab sistemi.